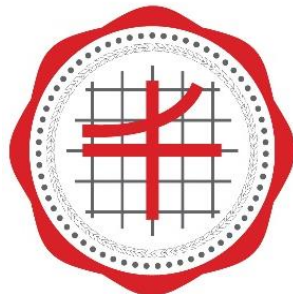


CPE 121 Mobile Application Developments

บทที่ 6

สายอักขระ (String)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.
Department of Computer Engineering

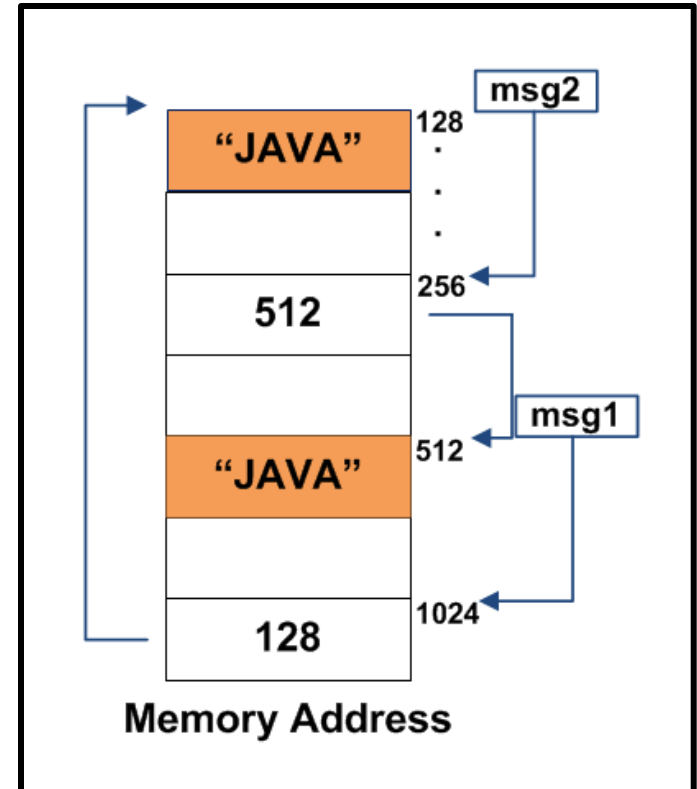
สายอักขระ

สายอักขระ (String) เป็นชนิดข้อมูลที่ประกอบด้วยตัวอักขระหลายๆ ตัวมาเรียงต่อกัน โดยเป็นการ **จองเพื่อเก็บค่าของตำแหน่ง** **อ้างอิง**ที่ใช้ในการเข้าถึงค่าของข้อมูล **ไม่ใช่จองเพื่อเก็บค่าของข้อมูล**



สายอักขระ

```
import java.util.Scanner;
public class StringTest {
    public static void main(String[] args) {
        String msg1 = "JAVA";
        System.out.print("Enter msg2 = ");
        Scanner scan = new Scanner(System.in);
        String msg2 = scan.nextLine();
        System.out.print("msg1 = " + msg1);
        System.out.println(" and msg2 = " + msg2);
        if (msg1 == msg2){
            System.out.println("msg1 == msg2");
        } else{ System.out.println("msg1 != msg2");
            msg1 = msg2; }
        System.out.print("msg1 = " + msg1);
        System.out.println(" and msg2 = " + msg2);
        if (msg1 == msg2){
            System.out.println("msg1 == msg2");
        } else{System.out.println("msg1 != msg2");
        }
    }
}
```



สายอักขระ

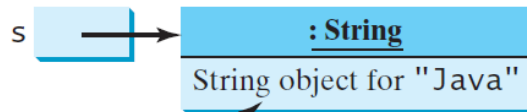
A **String** object is immutable: Its content cannot be changed once the string is created

```
String newString = new String(stringLiteral);
```

```
String message = new String("Welcome to Java");  
String message = "Welcome to Java";
```

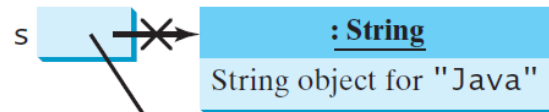
```
String s = "Java";  
s = "HTML";
```

After executing `String s = "Java";`



Contents cannot be changed

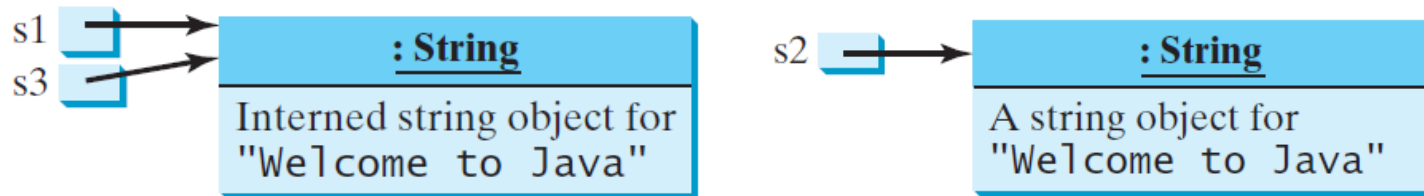
After executing `s = "HTML";`



This string object is now unreferences

สายอักขระ

```
String s1 = "Welcome to Java";  
String s2 = new String("Welcome to Java");  
String s3 = "Welcome to Java";  
System.out.println("s1 == s2 is " + (s1 == s2));  
System.out.println("s1 == s3 is " + (s1 == s3));
```



> run

s1 == s2 is false

s1 == s3 is true



คลาส String

java.lang.String

+equals(s1: Object): boolean
+equalsIgnoreCase(s1: String): Boolean

+compareTo(s1: String): int

+compareToIgnoreCase(s1: String): int

+regionMatches(index: int, s1: String, s1Index: int, len: int): boolean
+regionMatches(ignoreCase: boolean, index: int, s1: String, s1Index: int, len: int): Boolean

+startsWith(prefix: String): boolean
+endsWith(suffix: String): boolean

- Returns true if this string is equal to string s1.
- Returns true if this string is equal to string s1 case insensitive.
- Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
- Same as compareTo except that the comparison is case insensitive.
- Returns true if the specified subregion of this string exactly matches the specified subregion in string s1.
- Same as the preceding method except that you can specify whether the match is case sensitive.

- Returns true if this string starts with the specified prefix.
- Returns true if this string ends with the specified suffix.



เมธอด equals()

เมธอด **equals()** เป็นเมธอดที่ใช้เปรียบเทียบค่าข้อมูลชนิด String 2 ค่า

`result = str1.equals(str2);`

โดยที่

str1	เป็นข้อมูลชนิด String ตัวที่ 1
str2	เป็นข้อมูลชนิด String ตัวที่ 2
result	เป็นตัวแปรข้อมูลชนิด boolean

```
String msg1 = "Java";
String msg2 = "Java";
if(msg1.equals(msg2)){
    System.out.println("msg1 is equal to msg2")
}
else{
    System.out.println("msg1 is not equal to msg2")
}
```

> run StringTest

msg1 is equal to msg2

เมธอด compareTo()

เมธอด **compareTo()** เป็นเมธอดที่ใช้เปรียบเทียบค่าข้อมูลชนิด String 2 ค่า เช่นกัน

```
n = str1.compareTo(str2);
```

โดยที่

str1

เป็นข้อมูลชนิด String ตัวที่ 1

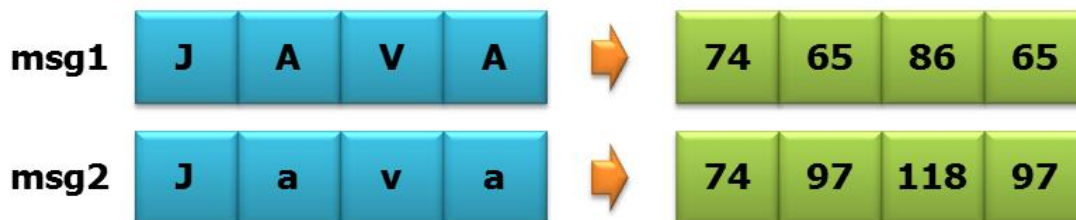
str2

เป็นข้อมูลชนิด String ตัวที่ 2

n

เป็นตัวแปรข้อมูลชนิด int เก็บค่าผลที่ได้จากการเปรียบเทียบข้อมูล

```
String msg1 = "JAVA";  
String msg2 = "Java";  
int n = msg1.compareTo(msg2)  
System.out.println("result is" + n)
```



> run StringTest
result is -32

เมธอด **startsWith()** / **endsWith()**

เมธอด **startsWith()** เป็นเมธอดที่ใช้สำหรับตรวจสอบข้อความส่วนหัวของข้อมูลชนิด String

```
result = str1. startsWith(str2);
```

เมธอด **endsWith()** เป็นเมธอดที่ใช้สำหรับตรวจสอบข้อความส่วนท้ายของข้อมูล

```
result = str1. endsWith(str2);
```

โดยที่

str1 เป็นข้อมูลชนิด String ที่ต้องการตรวจสอบ

str2 เป็นข้อมูลชนิด String ที่ใช้ตรวจสอบ

result เป็นตัวแปรข้อมูลชนิด boolean สำหรับรับค่าการเมธอด equals()

```
String msg = "java";  
System.out.println(msg.startWith("va"));  
System.out.println(msg.endsWith("va"));
```

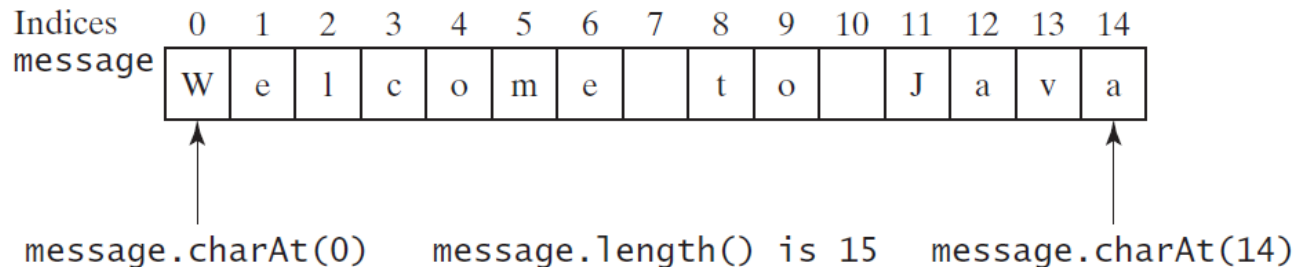
```
> run StringTest  
false  
true
```

คลาส String

java.lang.String

+length(): int
+charAt(index: int): char
+concat(s1: String): String

- Returns the number of characters in this string.
- Returns the character at the specified index from this string.
- Returns a new string that concatenates this string with string s1.



เมธอด length()

String เมธอด **length()** เป็นเมธอดที่ใช้สำหรับนับจำนวนตัวอักขระของข้อมูลชนิด String

```
n = str.length();
```

โดยที่

str

เป็นข้อมูลชนิด String ที่ต้องการนับจำนวนตัวอักขระ

n

เป็นตัวแปรข้อมูลชนิด int ที่รับค่าจำนวนตัวอักขระ

```
String msg = "java";  
System.out.println(msg.length());
```

```
> run StringTest
```

```
4
```



เมธอด `charAt()`

เมธอด `charAt()` เป็นเมธอดที่ใช้สำหรับค้นหาและแสดงค่าตัวอักษรในข้อมูลชนิด `String` ณ ตำแหน่งที่ต้องการ

```
ch = str.charAt(index);
```

โดยที่

`str` เป็นข้อมูลชนิด `String` ที่ต้องการค้นหาและแสดงค่าตัวอักษร

`index` เป็นตำแหน่งของข้อความที่ต้องการค้นหาตัวอักษร

`ch` เป็นตัวแปรข้อมูลชนิด `char` ที่รับค่าตัวอักษรจากเมธอด `charAt()`

```
String msg = "java";  
System.out.println(msg.charAt(2));
```

```
> run StringTest  
v
```



เมธอด concat()

เมธอด **concat()** เป็นเมธอดที่ใช้สำหรับเชื่อมต่อข้อมูลชนิด String หรือใช้เครื่องหมาย +

```
str3 = str1.concat(str2);  
str3 = str1+ str2;
```

โดยที่

str1	เป็นข้อมูลชนิด String ตัวที่ 1
str2	เป็นข้อมูลชนิด String ตัวที่ 2
str3	เป็นตัวแปรข้อมูลชนิด String เก็บค่าผลรวมของ str1 กับ str2

```
String msg1 = "Java";  
String msg2 = "Programming";  
String msg3 = "Language";  
String msg = msg1.concat(msg2);  
System.out.println(msg);  
msg = msg+msg3;  
System.out.println(msg);
```

> run StringTest

JAVA Programming

JAVA Programming Language



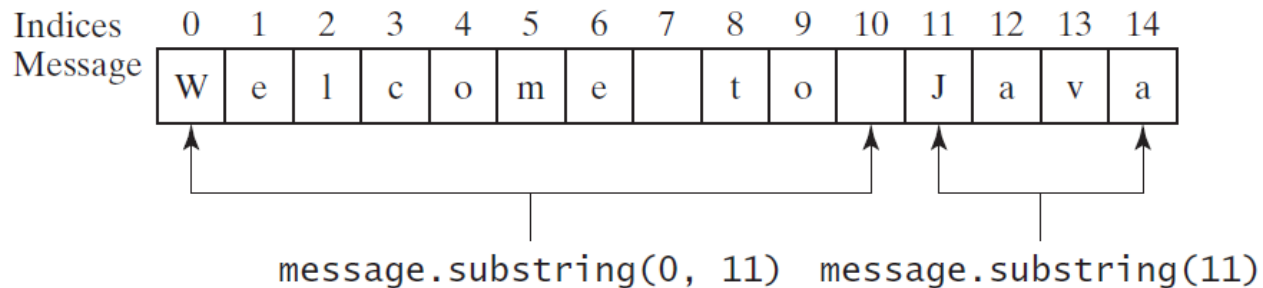
คลาส String

java.lang.String

+substring(beginIndex: int): String

+substring(beginIndex: int,
endIndex: int): String

- Returns this string's substring that begins with the character at the specified beginIndex and extends to the end of the string
- Returns this string's substring that begins at the specified beginIndex and extends to the character at index endIndex – 1. Note that the character at endIndex is not part of the substring.



เมธอด substring()

เมธอด **substring()** เป็นเมธอดที่ใช้สำหรับตัดข้อมูลชนิด String ให้เป็นข้อความย่อย

```
str2 = str1.substring(x, y);
```

โดยที่

str1

เป็นข้อมูลชนิด String ที่ต้องการตัด

str2

เป็นตัวแปรที่รับค่าข้อความที่ได้จากเมธอด substring()

x

เป็นตำแหน่งเริ่มต้นที่ต้องการตัดข้อความ

y

เป็นตำแหน่งสิ้นสุดของตัวอักขระที่ต้องการ

```
String msg = "Programming";  
msg = msg.substring(3,7);  
System.out.println(msg);
```

```
> run StringTest  
gram
```



คลาส String

java.lang.String

+toLowerCase(): String
+toUpperCase(): String
+trim(): String

+replace(oldChar: char, newChar: char): String
+replaceFirst(oldString: String, newString: String): String
+replaceAll(oldString: String, newString: String): String
+split(delimiter: String): String[]

- Returns a new string with all characters converted to lowercase.
- Returns a new string with all characters converted to uppercase.
- Returns a new string with whitespace characters trimmed on both sides.
- Returns a new string that replaces all matching characters in this string with the new character.
- Returns a new string that replaces the first matching substring in this string with the new substring.
- Returns a new string that replaces all matching substrings in this string with the new substring.
- Returns an array of strings consisting of the substrings split by the delimiter.



เมธอด toUpperCase() / toLowerCase()

เมธอด toUpperCase() เป็นเมธอดที่ใช้สำหรับเปลี่ยนข้อมูลให้เป็นตัวอักษรพิมพ์ใหญ่ทั้งหมด

```
str2= str1.toUpperCase();
```

โดยที่

str1 เป็นข้อมูลชนิด String ที่ต้องการเปลี่ยนเป็นตัวอักษรพิมพ์ใหญ่
str2 เป็นตัวแปรที่รับค่าข้อความที่ได้จากเมธอด toUpperCase()

เมธอด toLowerCase() เป็นเมธอดที่ใช้สำหรับเปลี่ยนข้อมูลให้เป็นตัวอักษรพิมพ์เล็กทั้งหมด

```
str2= str1.toLowerCase();
```

โดยที่

str1 เป็นข้อมูลชนิด String ที่ต้องการเปลี่ยนเป็นตัวอักษรพิมพ์เล็ก
str2 เป็นตัวแปรที่รับค่าข้อความที่ได้จากเมธอด toLowerCase()

```
String msg = "java";  
msg = msg.toUpperCase();  
System.out.println(msg);  
msg = msg.toLowerCase();
```

```
> run StringTest  
JAVA  
java
```

เมธอด `replace()`

เมธอด `replace()` เป็นเมธอดที่ใช้สำหรับแทนที่ส่วนของข้อความในข้อมูลชนิด String

```
str2 = str1.replace(str3, str4);
```

โดยที่

str1	เป็นข้อมูลชนิด String ที่จะถูกแทนที่ข้อความบางส่วนด้วยข้อความที่กำหนด
str2	เป็นตัวแปรที่รับค่าข้อความที่ได้จากเมธอด <code>replace()</code>
str3	เป็นข้อมูลชนิด String ที่จะถูกแทนที่ในข้อความเดิม
str4	เป็นข้อมูลชนิด String ที่จะเอาไปแทนที่ในข้อความเดิม

```
String msg = "JAVA Programming Language";  
msg = msg.replace("JAVA ", " C ");  
System.out.println(msg);
```

```
> run StringTest
```

```
C Programming Language
```

ตัวอย่างการใช้เมธอดในคลาส String

```
"Welcome".toLowerCase() >> returns a new string, welcome.  
"Welcome".toUpperCase() >> returns a new string, WELCOME.  
"\t Good Night \n".trim() >> returns a new string, Good Night.  
"Welcome".replace('e', 'A') >> returns a new string, WAlcomA.  
"Welcome".replaceFirst("e", "AB") >> returns a new string, WABlcome.  
"Welcome".replace("e", "AB") >> returns a new string, WABlcomAB.  
"Welcome".replace("el", "AB") >> returns a new string, WABcome.
```

toLowerCase()
toUpperCase()
trim()
replace
replaceFirst
replace
replace

```
String[] tokens = "Java#HTML#Perl".split("#");  
for (int i = 0; i < tokens.length; i++)  
System.out.print(tokens[i] + " ");
```

split

```
> run
```

```
Java HTML Perl
```



คลาส String

java.lang.String

+indexOf(ch: char): int
+indexOf(ch: char, fromIndex: int): int
+indexOf(s: String): int
+indexOf(s: String, fromIndex: int): int
+lastIndexOf(ch: int): int
+lastIndexOf(ch: int, fromIndex: int): int
+lastIndexOf(s: String): int
+lastIndexOf(s: String, fromIndex: int): int

- Returns the index of the first occurrence of **ch** in the string. Returns -1 if not matched.
- Returns the index of the first occurrence of **ch** after fromIndex in the string. Returns -1 if not matched.
- Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
- Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
- Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
- Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
- Returns the index of the last occurrence of string s. Returns -1 if not matched.
- Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.



เมธอด indexOf()

เมธอด indexOf() เป็นเมธอดที่ใช้สำหรับค้นหาและแสดงค่าตำแหน่งของตัวอักขระในข้อมูลชนิด String

```
index = str.indexOf(ch);
```

โดยที่

str

เป็นข้อมูลชนิด String ที่ต้องการค้นหาและแสดงค่าตำแหน่งของตัวอักขระ

ch

เป็นตัวอักขระที่ต้องการค้นหา

index

เป็นตำแหน่งของตัวอักขระที่ค้นหาพบใน str

ในกรณีที่ไม่มีพบตัวอักขระ ch เมธอดจะให้ค่าเป็นจำนวนเต็มลบ

```
String msg = "java";  
System.out.println(msg.indexOf('v'));  
System.out.println(msg.indexOf('A'));
```

```
> run StringTest
```

```
2
```

```
-1
```



เมธอด `lastIndexOf()`

เมธอด `lastIndexOf()` เป็นเมธอดที่ใช้สำหรับค้นหาและแสดงค่าตำแหน่งสุดท้ายที่พบตัวอักขระในข้อมูลชนิด `String`

โดยที่

<code>str</code>	เป็นข้อมูลชนิด <code>String</code> ที่ต้องการค้นหา
<code>ch</code>	เป็นอักขระที่ใช้ในการค้นหา
<code>index</code>	เป็นตำแหน่งของตัวอักขระ <code>ch</code> ที่ค้นหาพบใน <code>str</code> ในกรณีที่ไม่มีพบตัวอักขระ <code>ch</code> เมธอดจะให้ค่าเป็นจำนวนเต็ม ลบ

```
String msg = "java";  
System.out.println(msg.lastIndexOf('a'));  
System.out.println(msg.lastIndexOf('j'));
```

```
> run StringTest
```

```
3
```

```
-1
```



ตัวอย่างการใช้เมธอดในคลาส String

```
"Welcome to Java".indexOf('W') >> returns 0.  
"Welcome to Java".indexOf('o') >> returns 4.  
"Welcome to Java".indexOf('o', 5) >> returns 9.  
"Welcome to Java".indexOf("come") >> returns 3.  
"Welcome to Java".indexOf("Java", 5) >> returns 11.  
"Welcome to Java".indexOf("java", 5) >> returns -1.
```

indexOf

```
"Welcome to Java".lastIndexOf('W') >> returns 0.  
"Welcome to Java".lastIndexOf('o') >> returns 9.  
"Welcome to Java".lastIndexOf('o', 5) >> returns 4.  
"Welcome to Java".lastIndexOf("come") >> returns 3.  
"Welcome to Java".lastIndexOf("Java", 5) >> returns -1.  
"Welcome to Java".lastIndexOf("Java") >> returns 11.
```

lastIndexOf



ตัวอย่างการประยุกต์ใช้เมธอดในคลาส String

```
import java.util.Scanner;
import java.util.ArrayList;
class product_detail {
    String pcode, date, id, size, name; int n;
    ArrayList code = new ArrayList();
    public void inputdata() {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter product code => ddmmyyyy0000MBreeze :\n"+
        " (Press x to Exit) ");
        do { System.out.print("product code : ");
            pcode = scan.nextLine();
            if (! (pcode.equals("x"))) {
                code.add(pcode);    }
        } while (!(pcode.equals("x")));
    }
}
```

ตัวอย่างการประยุกต์ใช้เมธอดในคลาส String

```
public void showdata() {  
    if (code.size() > 0) {  
        System.out.printf("%-8s%-6s%-5s%-20s\n", "date", "id", "size", "name");  
        for (int i=0; i<code.size(); i++) {  
            date = getinfo(code.get(i).toString(),0,8);  
            id  = getinfo(code.get(i).toString(),8,12);  
            size = getinfo(code.get(i).toString(),12,13);  
            name = getinfo(code.get(i).toString(),13, code.get(i).toString().length());  
            System.out.printf("%-8s%-6s%-5s%-10s\n",date,id,size,name); }  
        } }  
    public String getinfo(String code, int a, int b) {  
        String info = code.substring(a,b);  
        return info; }  
}
```

```
public class ProductCode {  
    public static void main(String[] args) {  
        product_detail product = new product_detail();  
        product.inputdata();  
        product.showdata(); }  
}
```

ตัวอย่างการประยุกต์ใช้เมธอดในคลาส String

> run ProductCode

Enter product code => ddmmyyyy0000MBreeze :

(Press x to Exit)

product code : **250720160000SCoke**

product code : **250720160000MPepsi**

product code : **250720160000FFanta**

product code : **x**

date id size name

25072016 0000 S Coke

25072016 0000 M Pepsi

25072016 0000 F Fanta



คลาส String

java.lang.String

+valueOf(c: char): String
+valueOf(data: char[]): String
+valueOf(d: double): String
+valueOf(f: float): String
+valueOf(i: int): String
+valueOf(l: long): String
+valueOf(b: boolean): String

- Returns a string consisting of the character c.
- Returns a string consisting of the characters in the array.
- Returns a string representing the double value.
- Returns a string representing the float value.
- Returns a string representing the int value.
- Returns a string representing the long value.
- Returns a string representing the boolean value.

For example, to convert a **double** value **5.44** to a string, use `String.valueOf(5.44)`.
The return value is a string consisting of the characters '5', '.', '4', and '4'.



ตัวอย่างการแปลง Hexadecimals to Decimals

```
import java.util.Scanner;

public class Hexadecimal2Decimal {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a hex number: ");
        String hex = input.nextLine();
        System.out.println("The decimal value for hex number " + hex + " is " +
            hexToDecimal(hex.toUpperCase()));
    }

    public static int hexToDecimal(String hex) {
        int decimalValue = 0;
        for (int i = 0; i < hex.length(); i++) {
            char hexChar = hex.charAt(i);
            decimalValue = decimalValue * 16 + hexCharToDecimal(hexChar);
        }
        return decimalValue;
    }
}
```

ตัวอย่างการแปลง Hexadecimals to Decimals

```
public static int hexCharToDecimal(char ch) {  
    if (ch >= 'A' && ch <= 'F') {  
        return 10 + ch - 'A';  
    }  
    else { // ch is '0', '1', ..., or '9'  
        return ch - '0';  
    }  
}
```

> run

Enter a hex number: AB8C

The decimal value for hex number AB8C is 43916

> run

Enter a hex number: af71

The decimal value for hex number af71 is 44913



คลาส Character

java.lang.Character

```
+Character(value: char)
+charValue(): char
+compareTo(anotherCharacter: Character): int
+equals(anotherCharacter: Character): boolean
+isDigit(ch: char): boolean
+isLetter(ch: char): boolean
+isLetterOrDigit(ch: char): boolean
+isLowerCase(ch: char): boolean
+isUpperCase(ch: char): Boolean
+toLowerCase(ch: char): char
+toUpperCase(ch: char): char
```

- Constructs a character object with char value.
- Returns the char value from this object.
- Compares this character with another.
- Returns true if this character is equal to another.
- Returns true if the specified character is a digit.
- Returns true if the specified character is a letter.
- Returns true if the character is a letter or a digit.
- Returns true if the character is a lowercase letter.
- Returns true if the character is an uppercase letter.
- Returns the lowercase of the specified character.
- Returns the uppercase of the specified character.

```
Character charObject = new Character('b');
```

```
charObject.compareTo(new Character('a')) >> returns 1
charObject.compareTo(new Character('b')) >> returns 0
charObject.compareTo(new Character('c')) >> returns -1
charObject.compareTo(new Character('d')) >> returns -2
charObject.equals(new Character('b')) >> returns true
charObject.equals(new Character('d')) >> returns false
```

คลาส Character

The **isDigit(char ch)** method >> returns **true** if the character is a digit.

The **isLetter(char ch)** method >> returns **true** if the character is a letter.

The **isLetterOrDigit(char ch)** method >> returns **true** if the character is a letter or a digit.

The **isLowerCase(char ch)** method >> returns **true** if the character is a lowercase letter

The **isUpperCase(char ch)** method >> returns **true** if the character is an uppercase letter.

The **toLowerCase(char ch)** method >> returns the lowercase letter for the character

The **toUpperCase(char ch)** method >> returns the uppercase letter for the character.



ตัวอย่างการประยุกต์ใช้คลาส Character

```
import java.util.Scanner;
public class CountEachLetter {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = input.nextLine();
        int[] counts = countLetters(s.toLowerCase());
        // Display results
        for (int i = 0; i < counts.length; i++) {
            if (counts[i] != 0) {
                System.out.println((char)('a' + i) + " appears " + counts[i] +
                    ((counts[i] == 1) ? " time" : " times"));
            }
        }
    }
}
```

ตัวอย่างการประยุกต์ใช้คลาส Character

```
/** Count each letter in the string */  
public static int[] countLetters(String s) {  
    int[] counts = new int[26];  
    for (int i = 0; i < s.length(); i++) {  
        if (Character.isLetter(s.charAt(i))) {  
            counts[s.charAt(i) - 'a']++;  
        }  
    }  
    return counts;  
}
```

> run

Enter a string: abababx

a appears 3 times

b appears 3 times

x appears 1 time



คลาส StringBuffer และ คลาส StringBuilder

คลาส **StringBuffer** มีลักษณะการใช้งานเหมือนคลาส String แต่มีคุณสมบัติบางอย่างที่แตกต่างจากคลาส String คือ

- String ไม่สามารถเปลี่ยนแปลงค่าที่อยู่ภายในได้หลังจากที่ถูกสร้างขึ้นมาแล้ว ในขณะที่ StringBuffer ออกแบบมาให้สามารถแก้ไขเปลี่ยนแปลงค่าได้
- การใช้เมธอดเปลี่ยนแปลงค่า String จะเป็นการสร้าง String ตัวใหม่ ทำให้ประสิทธิภาพการทำงานของโปรแกรมลดลงได้ ในขณะที่ StringBuffer มีเมธอดเปลี่ยนแปลงค่าในตัวเอง โดยไม่ต้องสร้างชิ้นใหม่การทำงานจึงมีประสิทธิภาพมากกว่า

ดังนั้น ในกรณีที่มีการเปลี่ยนแปลงหรือแก้ไขค่าของข้อความควรใช้ StringBuffer แทน String

คลาส **StringBuilder** มีลักษณะการใช้งานเหมือนคลาส StringBuffer แต่มีคุณสมบัติที่แตกต่างจากคลาส StringBuffer คือ **คลาส StringBuffer มีการทำงานที่สับสนและสร้างความปลอดภัยได้มากกว่าแต่จะทำงานได้ช้ากว่าคลาส StringBuilder**



เมธอดในคลาส StringBuffer และ คลาส StringBuilder

เมธอด **append()** ใช้สำหรับเพิ่มความแบบต่อท้ายข้อมูลเดิม

```
str3 = str1.append(str2);
```

โดยที่

str1 เป็นข้อมูลชนิด StringBuffer ที่ต้องการเพิ่มความต่อท้าย

str2 เป็นข้อมูลชนิด StringBuffer ที่ต้องการนำไปต่อท้าย str1

str3 เป็นข้อมูลชนิด StringBuffer สำหรับรับค่าข้อมูลที่ได้จากต่อข้อความ

เมธอด **insert()** ใช้สำหรับข้อความแทรกระหว่างข้อมูลเดิม ณ ตำแหน่งที่ต้องการ

```
str3= str1. insert(index, str2);
```

โดยที่

str1 เป็นข้อมูลชนิด StringBuffer ที่ต้องการแทรกข้อความ

str2 เป็นข้อมูลชนิด StringBuffer ที่ต้องการแทรกใน str1

str3 เป็นข้อมูลชนิด StringBuffer สำหรับรับค่าจากเมธอด insert()

index เป็นตำแหน่งที่ต้องการแทรก str2 ใน str1



เมธอดในคลาส StringBuffer และ คลาส StringBuilder

เมธอด **delete()** ใช้ลบส่วนของข้อความในข้อมูล ณ ตำแหน่งที่ต้องการ

```
str2 = str1.delete(x, y);
```

โดยที่

str1 เป็นข้อมูลชนิด StringBuffer ที่ต้องการลบข้อความ

str2 เป็นตัวแปรข้อมูลชนิด StringBuffer สำหรับรับค่าข้อมูล str1 ที่ลบข้อความแล้ว

x เป็นตำแหน่งเริ่มต้นของตัวอักขระที่ต้องการลบ

y เป็นตำแหน่งสิ้นสุดของตัวอักขระที่ต้องการลบ

เมธอด **length()** ใช้นับจำนวนตัวอักขระของข้อมูล

```
int n = str.length();
```

โดยที่

str เป็นข้อมูลชนิด StringBuffer ที่ต้องการนับจำนวนตัวอักขระ

n เป็นตัวแปรสำหรับรับผลการนับจำนวนตัวอักขระ



โปรแกรมการใช้งานเมธอดในคลาส **StringBuffer** และ คลาส **StringBuilder**

```
public class StringMethod {  
    public static void main(String[] args) {  
        String name1 = "Smith";  
        name1 = name1.concat("Brown");  
        System.out.println("===== String =====");  
        System.out.println("name1 = \"\" + name1 + \"\"");  
        StringBuffer name2 = new StringBuffer("Smith");  
        name2.append("Brown");  
        System.out.println("===== StringBuffer =====");  
        System.out.println("name2 = \"\" + name2 + \"\"");  
        System.out.println("===== String Method =====");  
        System.out.println("name1 = \"\" + name1 + \"\"");  
        System.out.println("name1.replace(\"h\", \"e\") = \" + name1.replace("h", "e"));  
        System.out.println("=====");  
    }  
}
```

โปรแกรมการใช้งานเมธอดในคลาส **StringBuffer** และ คลาส **StringBuilder**

```
System.out.println("===== StringBuffer Method =====");
System.out.println("name2 = \"" + name2 + "\"");
System.out.println("name2.insert(5,\"y\") = " + name2.insert(5,"y"));
System.out.println("=====");
System.out.println("===== StringBuilder Method =====");
StringBuilder name3 = new StringBuilder(name2);
System.out.println("name3 = \"" + name3 + "\"");
System.out.println("name3.delete(4,5)= " + name3.delete(4,5));
System.out.println("=====");
}
}
```



โปรแกรมการใช้งานเมธอดในคลาส **StringBuffer** และ คลาส **StringBuilder**

> run StringMethod

===== String =====

name1 = "Smith Brown"

===== StringBuffer =====

name2 = "Smith Brown"

===== String Method =====

name1 = "Smith Brown"

name1.replace("h","e")= Smite Brown

=====

===== StringBuffer Method =====

name2 = "Smith Brown"

name2.insert(5,"y") = Smithy Brown

=====

===== StringBuider Method =====

name3 = "Smithy Brown"

name3.delete(4,5)= Smity Brown

=====